

1. Energy Monitor (Default Method Conflict Resolution)

```
import java.util.*;

interface SolarPowered {
    default double calculateCostPerUnit() {
        return 2.5;
    }
}

interface GridPowered {
    default double calculateCostPerUnit() {
        return 7.0;
    }
}

// HybridHome class implementing both interfaces
class HybridHome implements SolarPowered, GridPowered {

    private double solarPercentage;
    private double gridPercentage;

    // Default Constructor
    public HybridHome() {
        solarPercentage = 50;
        gridPercentage = 50;
    }

    // Parameterized Constructor
    public HybridHome(double solarPercentage, double gridPercentage) {
        this.solarPercentage = solarPercentage;
        this.gridPercentage = gridPercentage;
    }

    // Overriding default method to resolve ambiguity
    public double calculateCostPerUnit() {
        double solarCost = SolarPowered.super.calculateCostPerUnit();
        double gridCost = GridPowered.super.calculateCostPerUnit();

        return (solarPercentage/100 * solarCost) +
            (gridPercentage/100 * gridCost);
    }

    // Monthly bill calculation
    public double monthlyBill(double units) {
        return units * calculateCostPerUnit();
    }

    // Carbon footprint calculation
    public double carbonFootprint(double units) {
        double gridUnits = (gridPercentage/100) * units;
        return gridUnits * 0.82; // kg CO2 per unit
    }

    // toString method
```

```

    public String toString() {
        return "Solar Usage: " + solarPercentage + "%\n" +
            "Grid Usage: " + gridPercentage + "%\n" +
            "Cost per Unit: " + calculateCostPerUnit();
    }
}

// Main class
public class EnergyMonitor {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter total units consumed: ");
        double units = sc.nextDouble();

        System.out.print("Enter solar usage percentage: ");
        double solar = sc.nextDouble();

        double grid = 100 - solar;

        // Object creation through user input
        HybridHome h = new HybridHome(solar, grid);

        System.out.println("\n--- Energy Consumption Details ---");
        System.out.println(h);

        System.out.println("Monthly Bill: " + h.monthlyBill(units));
        System.out.println("Carbon Footprint: " + h.carbonFootprint(units) + " kg CO2");

        sc.close();
    }
}

```

```

[annkurr@archlinux Q1]$ java EnergyMonitor
Enter total units consumed: 500
Enter solar usage percentage: 10

--- Energy Consumption Details ---
Solar Usage: 10.0%
Grid Usage: 90.0%
Cost per Unit: 6.55
Monthly Bill: 3275.0
Carbon Footprint: 369.0 kg CO2

```

2. Smart City Grid (Complex Interface Hierarchy)

```

import java.util.*;

// ----- INTERFACES -----

// PowerGenerable

```

```

interface PowerGenerable {
    double generatePower(double inputResource);

    default double transmissionLoss(double generatedKWh, double distanceKm) {
        return generatedKWh * 0.03 * Math.sqrt(distanceKm);
    }

    default double carbonCredit(double renewablePercent, double totalKWh) {
        return (renewablePercent / 100) * totalKWh * 0.82;
    }
}

// RenewableGenerable
interface RenewableGenerable extends PowerGenerable {

    double capacityFactor(double actualOutput, double ratedCapacity);

    default double solarOutput(double panelArea, double irradiance, double efficiency) {
        return panelArea * irradiance * efficiency;
    }

    default double windOutput(double radius, double windSpeed, double efficiency) {
        return 0.5 * 1.225 * Math.PI * radius * radius *
            Math.pow(windSpeed, 3) * efficiency;
    }
}

// StorageCapable
interface StorageCapable extends RenewableGenerable {

    void chargeBattery(double excessKWh);

    default double batteryEfficiency(double stored, double retrieved) {
        return (retrieved / stored) * 100;
    }

    default double optimalDischargeTime(double storedKWh, double peakKW) {
        return storedKWh / peakKW;
    }
}

// GridManageable
interface GridManageable {

    double regulateVoltage(double currentLoad);

    default double gridFrequency(double P, double Q) {
        return P / Math.sqrt(P * P + Q * Q);
    }

    default double peakDemandSurcharge(double peakKW, double avgKW) {
        return (peakKW > 1.3 * avgKW) ? peakKW * 0.1 : 0;
    }
}

```

```

// LoadBalanceable
interface LoadBalanceable extends GridManageable {

    void redistributeLoad(double[] zoneLoads);

    default double loadVariance(double[] loads) {
        double mean = 0;
        for (double l : loads) mean += l;
        mean /= loads.length;

        double var = 0;
        for (double l : loads)
            var += Math.pow(l - mean, 2);

        return var / loads.length;
    }

    default double spilloverRisk(double supply, double demand) {
        return demand / supply;
    }
}

// SmartMeteringEnabled
interface SmartMeteringEnabled extends LoadBalanceable {

    void readMeterData(String zoneId);

    default double timeOfUseBilling(double kWh, String slot) {
        if (slot.equalsIgnoreCase("off"))
            return kWh * 4.2;
        else if (slot.equalsIgnoreCase("shoulder"))
            return kWh * 6.8;
        else
            return kWh * 11.5;
    }

    default double demandResponseIncentive(double reducedKWh) {
        return reducedKWh * 3.5;
    }
}

// ----- CLASSES -----

// Infrastructure
class Infrastructure {
    protected String facilityId, location;
    protected int year;
    protected double operationalCost;

    public Infrastructure() {}

    public Infrastructure(String id, String loc, int y, double cost) {
        facilityId = id;

```

```

        location = loc;
        year = y;
        operationalCost = cost;
    }

    public double depreciatedValue(double originalCost, int years) {
        return originalCost * (1 - 0.05 * years);
    }

    public double maintenanceCost(double value) {
        return value * 0.025;
    }
}

// PowerStation
class PowerStation extends Infrastructure implements GridManageable, PowerGenerable {

    protected double installedCapacity;
    protected double currentLoad;

    public PowerStation() {}

    public PowerStation(String id, String loc, int y, double cost,
                        double cap, double load) {
        super(id, loc, y, cost);
        installedCapacity = cap;
        currentLoad = load;
    }

    public double generatePower(double input) {
        return input * 0.35; // thermal efficiency
    }

    public double regulateVoltage(double load) {
        double nominal = 230;
        double droop = 5;
        return (nominal - load) / droop;
    }

    public double efficiencyRating(double actualOutput) {
        return (actualOutput / installedCapacity) * 100;
    }
}

// SmartRenewableGrid
class SmartRenewableGrid extends PowerStation
    implements StorageCapable, SmartMeteringEnabled {

    double batteryStorage = 200000; // kWh
    double storedEnergy = 0;

    public SmartRenewableGrid(String id, String loc, int y, double cost,
                            double cap, double load) {
        super(id, loc, y, cost, cap, load);
    }
}

```

```

}

// override transmissionLoss (15% improvement)
public double transmissionLoss(double gen, double dist) {
    double base = StorageCapable.super.transmissionLoss(gen, dist);
    return base * 0.85;
}

public double capacityFactor(double actual, double rated) {
    return actual / rated;
}

public void chargeBattery(double excess) {
    if (excess > 0.1 * currentLoad)
        storedEnergy += excess;
}

public void redistributeLoad(double[] loads) {
    double total = 0;
    for (double l : loads) total += l;

    double equal = total / loads.length;
    for (int i = 0; i < loads.length; i++)
        loads[i] = equal;
}

public void readMeterData(String zoneId) {
    System.out.println("Reading meter for zone: " + zoneId);
}

// ----- CORE COMPUTATION -----
public void computeSystem() {

    // Solar: 50MW, 8 hrs, 18%
    double solar = 50 * 1000 * 8 * 0.18;

    // Wind: simplified
    double wind = 30 * 1000 * 24 * 0.42;

    double totalGen = solar + wind;

    double loss = transmissionLoss(totalGen, 50);

    double net = totalGen - loss;

    chargeBattery(net - currentLoad);

    double revenue =
        timeOfUseBilling(net * 0.5, "off") +
        timeOfUseBilling(net * 0.3, "shoulder") +
        timeOfUseBilling(net * 0.2, "peak");

    double carbon = carbonCredit(100, totalGen);

```

```

double dep = depreciatedValue(1_000_000, 5);
double maint = maintenanceCost(dep);

double profit = revenue - (maint + operationalCost);

System.out.println("\n--- SMART GRID REPORT ---");
System.out.println("Solar Generation: " + solar + " kWh");
System.out.println("Wind Generation: " + wind + " kWh");
System.out.println("Total Generation: " + totalGen);
System.out.println("Transmission Loss: " + loss);
System.out.println("Net Energy: " + net);
System.out.println("Revenue: " + revenue);
System.out.println("Carbon Credits: " + carbon);
System.out.println("Profit: " + profit);
    }
}

// ----- MAIN -----

public class SmartCityGrid {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Facility ID: ");
        String id = sc.next();

        System.out.print("Enter Location: ");
        String loc = sc.next();

        System.out.print("Enter Year: ");
        int year = sc.nextInt();

        System.out.print("Enter Operational Cost: ");
        double cost = sc.nextDouble();

        System.out.print("Enter Capacity (MW): ");
        double cap = sc.nextDouble();

        System.out.print("Enter Current Load (kW): ");
        double load = sc.nextDouble();

        SmartRenewableGrid grid =
            new SmartRenewableGrid(id, loc, year, cost, cap, load);

        grid.computeSystem();

        sc.close();
    }
}

```

```
[annkurr@archlinux Q2]$ java SmartCityGrid
```

```
Enter Facility ID: SG101
```

```
Enter Location: Goa
```

```
Enter Year: 2025
```

```
Enter Operational Cost: 50000
```

```
Enter Capacity (MW): 100
```

```
Enter Current Load (kW): 200000
```

```
--- SMART GRID REPORT ---
```

```
Solar Generation: 72000.0 kWh
```

```
Wind Generation: 302400.0 kWh
```

```
Total Generation: 374400.0
```

```
Transmission Loss: 67508.89861344207
```

```
Net Energy: 306891.1013865579
```

```
Revenue: 1976378.692929433
```

```
Carbon Credits: 307008.0
```

```
Profit: 1907628.692929433
```